



(/rol/app/)

Menu

Korean

홈 (/rol/app/) (home) > Introducing OpenShift Service Mesh

> Chapter 1: Introducing OpenShift Service Mesh

Introducing OpenShift Service Mesh

Lesson

Lab Environment

Service Mesh Showroom Application

Objectives

- Describe how the Bookinfo application provides a realistic scenario for demonstrating OpenShift Service Mesh features, including traffic routing, observability, and security.

About the Bookinfo Application

The Bookinfo application is polyglot. It is composed of four microservices that are written in different programming languages. This application is a realistic sample that you can use to learn about and explore Red Hat OpenShift Service Mesh features. The application displays information about a book, similar to a single catalog entry in an online bookstore.

Bookinfo Architecture

The Bookinfo application consists of four separate microservices, each deployed as a service within the service mesh. These services are independent of OpenShift Service Mesh. However, their architecture makes them an excellent example of demonstrating service mesh features, such as traffic management, observability, and security.

When you deploy the application, each microservice is packaged with an Envoy sidecar proxy. This proxy intercepts all incoming and outgoing calls for the services. This interception provides the necessary hooks for external control through the service mesh control plane.

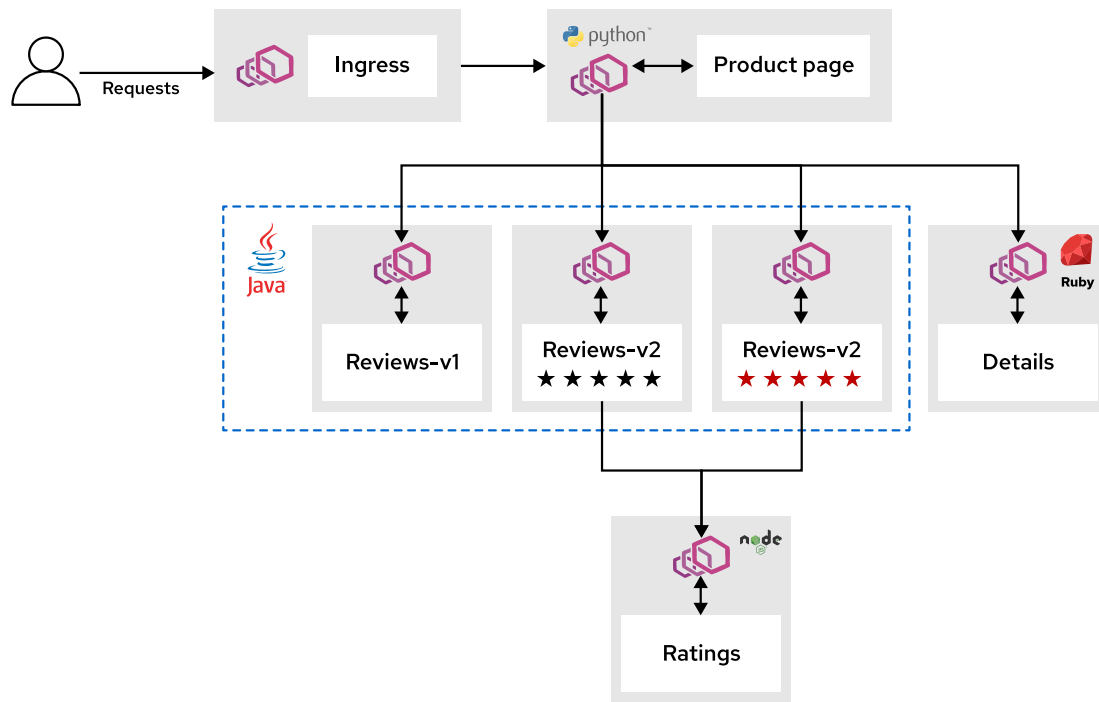


Figure 1.9: Bookinfo Application Architecture

The application comprises of four main microservices that collaborate to deliver the final web page.

Table 1.1. Bookinfo Microservice Details

Service Name	Language	Description
productpage	Python	The user-facing web page. Calls the details and reviews services to populate the page with content.
details	Ruby	Provides the book's metadata, including the ISBN, number of pages, and publisher.
reviews	Java	Provides the book review information. This service has multiple versions and calls the ratings service.
ratings	Node.js	Provides the rating information that accompanies a book review, such as a star rating.

The reviews Microservice Versions

The reviews microservice has three distinct versions. This multiversion deployment is crucial for demonstrating traffic management features, such as canary deployments and A/B testing. You can control which version of the reviews service a user sees by applying OpenShift Service Mesh traffic rules.

Table 1.2. The reviews Service Versions

Version	Behavior	Rating Display
v1	Does not call the ratings service.	No stars are displayed.
v2	Calls the ratings service.	Displays ratings as 1 to 5 black stars.
v3	Calls the ratings service.	Displays ratings as 1 to 5 red stars.

Service Mesh Features and Bookinfo

The structure of the Bookinfo application makes it an ideal tool for understanding the value of a service mesh. Its combination of multiple services, languages, and versions creates a scenario where managing service interaction is complex. This course leverages the Bookinfo application to demonstrate the OpenShift Service Mesh functionalities, such as the following:

Traffic Management

OpenShift Service Mesh uses `VirtualService` and `DestinationRule` custom OpenShift resources to control the flow of traffic. You can use these resources to configure sophisticated routing scenarios for the Bookinfo application. For example, you can direct a percentage of traffic to a new version of the reviews service for a canary release. You can also direct any tagged traffic, such as bearing a specific request header, to a corresponding version of a service.

Observability

Observability is a key feature of a service mesh. The Envoy sidecar proxies collect telemetry data for all service-to-service communication.

The Kiali console plug-in provides a powerful visualization of a service mesh. It generates a topology graph of the services, illustrating their connections and the flow of traffic between them. You can inspect service health, traffic rates, and the impact of the routing rules directly from the graph.

The distributed tracing implementation uses Grafana Tempo as the interface and provides the ability to trace requests. Tracing enables following a single request from the productpage service through the reviews service and to the ratings service. This detailed view helps you understand latency bottlenecks and diagnose errors in your microservices architecture.

You might explore the following Bookinfo observability characteristics:

- *Multi-service Tracing*: Requests flow through multiple services, creating rich trace data that shows service dependencies and call patterns.
- *Version-based Monitoring*: The multiple versions of the reviews service enable you to monitor and compare performance across different service versions.

- *Real-world Scenarios*: The application's realistic service interactions demonstrate how observability tools work with actual microservice communication patterns.

Security

OpenShift Service Mesh can secure communication between the Bookinfo microservices without requiring any changes to the application code. By applying a `PeerAuthentication` policy, you can enforce mutual TLS (mTLS) for all traffic within the mesh. When mTLS is enabled, Kiali displays a lock icon on the edges of its graph, indicating that the traffic is encrypted and secure.

IMPORTANT

Enforcing mTLS is a critical step for achieving a zero-trust network model. It ensures that all communication between services is authenticated and encrypted, which prevents man-in-the-middle attacks.

Generate Traffic to the Service Mesh

To effectively demonstrate and observe service mesh features, you need to generate traffic to the application in the mesh. This course uses a traffic generator script, `traffic_gen.py`, a custom Python-based tool that automates the process of sending HTTP requests to the mesh services.

Purpose of the Traffic Generator

The traffic generator provides the following functionalities:

- *Observability Data*: Continuous traffic generation enables the service mesh to collect telemetry data that you can visualize in Kiali and trace in Grafana Tempo.
- *Traffic Pattern Testing*: You can test routing rules, canary deployments, and other traffic management features by generating specific request patterns.
- *Performance Analysis*: The tool provides statistics such as success rates, response times, and percentile metrics to help you understand service behavior by identifying timeouts, retries, and other performance issues.

Running the Traffic Generator Tool

The traffic generator uses YAML configuration files that define traffic scenarios for each lab exercise. Each configuration file specifies the endpoint to target, the traffic pattern to generate, and any custom headers for routing tests.

The traffic generator supports the following traffic patterns:

- *Continuous*: Sends requests continuously for a specified duration, ideal for generating observability data in Kiali and Grafana Tempo.
- *Finite*: Sends a fixed number of requests, perfect for testing specific scenarios such as canary deployments.

- *Multi-scenario*: Runs multiple traffic patterns in sequence, useful for comprehensive testing.
- *Mix*: Interleaves requests across different endpoints by using round-robin, random, or weighted selection patterns.

To run the traffic generator script, you must execute the following command:

```
[user@host ~]$ traffic_gen.py config-file.yaml 1
```

- 1 Replace `config-file.yaml` with the appropriate configuration file for your lab exercise.

NOTE

Throughout this course, you use preconfigured YAML files tailored to each lab exercise. You do not need to create or modify these configuration files. Run the traffic generator with the specified configuration file as instructed in each lab.

The tool automatically discovers the Service Mesh gateway URL and begins sending requests. It displays real-time progress and provides summary statistics at the end of the test, including success rates, response distribution, and timing percentiles.

Understanding the Output

When you run the traffic generator, you see different types of output that help you understand what is happening:

```
[user@host ~]$ traffic_gen.py continuous.yaml
```

```
Continuous mode: 600s to http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
```

```
curl -s http://istio-ingressgateway-istio-ingress.apps.ocp4.example.com/reviews/1
```

```
[1] ✓ HTTP 200 -- red (0.0s)
[2] ✓ HTTP 200 -- No stars (0.1s)
[3] ✓ HTTP 200 -- black (0.4s)
[4] ✓ HTTP 200 -- black (0.5s)
[5] ✓ HTTP 200 -- black (0.6s)
[6] ✓ HTTP 200 -- No stars (0.8s)
[7] ✓ HTTP 200 -- No stars (0.9s)
[8] ✓ HTTP 200 -- red (1.0s)
[9] ✓ HTTP 200 -- black (1.1s)
```

```
...output omitted...
```

```
Interrupted by user (Ctrl+C). Stopping continuous mode...
```

Traffic Statistics

Total Request	Success Rate	Average	P50	P95
39	100.0% (39/39)	23.1ms	16.0ms	79.8ms

Response Distribution

Response	Count	Percentage
No stars	15	38.5%
black	14	35.9%
red	10	25.6%

- 1 The tool displays the equivalent `curl` command for each request, making it easy to reproduce requests manually if needed.
- 2 Each request shows a status indicator, HTTP status code, response content preview, and response time.
- 3 At the end of the test, you receive a comprehensive summary.
- 4 Total requests sent, Success rate percentage, and Average, P50, and P95 response times.
- 5 Response distribution, showing which service versions responded.
- 6 Error summary, if any failures occurred.

The tool prints colored text and clear formatting to facilitate easy interpretation of the results. This information helps you verify that your service mesh configuration is working as expected and provides insights into service performance and behavior.

You can always interrupt the traffic generator by pressing **Ctrl+C**.

REFERENCES

For more information about the Bookinfo application, refer to the *About the Bookinfo Application* section in the *Installing OpenShift Service Mesh* chapter in the Red Hat OpenShift Service Mesh 3.1 *Installing* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-about-bookinfo-application_ossm-installing-openshift-service-mesh

(https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-about-bookinfo-application_ossm-installing-openshift-service-mesh)

Istio.io: Bookinfo Application (<https://istio.io/v1.26/docs/examples/bookinfo/>)

GitHub: Bookinfo Application source code

(<https://github.com/istio/istio/tree/1.26.5/samples/bookinfo>)

Revision: ad0006l-3.1-ed3-20260112-15701ce



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

릴리스 노트

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Red Hat 교육 정책

(<https://www.redhat.com/ko/about/red-hat-training-policies>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

쿠키 기본설정